

Evaluating Jataayu

A Multi-Benchmark Study of a Runtime Authorization Layer for Tool-Using Agents

Sai Krishna Rallabandi
Jataayu Project
srallaba.research@gmail.com

July 2026 · Technical Report · Jataayu v0.3.0

Abstract

Jataayu is a runtime authorization layer for tool-using LLM agents. Its thesis: gate the agent’s *action* by the *provenance* of its input rather than classify attack text, because detection is evadable by paraphrase and character perturbation, and training-based defenses are bound to the model they retrain.

It guards seven distinct security surfaces: (1) a deterministic effect boundary (action authorization); (2) inbound injection screening; (3) tool-return injection screening; (4) memory-poisoning screening (read and write); (5) outbound PII redaction; (6) an egress-channel guard against zero-click exfiltration; and (7) skill supply-chain vetting. We evaluate each surface against a matched benchmark, public where one exists (AgentDojo, InjecAgent, deepset / safe-guard / jailbreak injection sets, ai4privacy) and synthetic-but-deterministic where none does, and report every number, including the ones that expose weaknesses.

Two findings are structural. First, the effect boundary works: on an AgentDojo-derived action stream it lifts attack prevention from 29% to **100%** once the tool→effect map is complete, at a disclosed utility cost. A direct semantic-vs-effect probe validates the thesis: a text grader catches only the 1 overt trajectory of 6 harmful (0 of 5 silent harms; recall 0.17), while the effect grader catches all six (**1.00**). Second, the evaluation is honest. It surfaced four concrete defects: a shipped coverage gap that silently authorized every banking, travel, and Slack side-effect; the absence of a deferred-instruction detector (sleeper-memory recall 0.00); an over-narrow exfiltration key in the composition guard; and a live regex false-positive. One of these we fix in-line (disclose-then-fix). The deterministic core runs in tens of microseconds per decision (0.036 ms mean).

1 Introduction

An agent that reads the web, email, or a ticket consumes attacker-controlled text. Indirect prompt injection exploits the model’s inability to separate trusted instructions from untrusted data, and the problem is not close to solved: on Agent Security Bench the strongest attack (a mixed direct+indirect+memory-poisoning attack) averages 84.3% success across agents and indirect prompt injection alone reaches 4.8–59.7% depending on the backbone, with the paper explicitly finding that current defenses are insufficient (arXiv:2410.02644v3, Table 5 [14]); the canonical InjecAgent baseline puts an undefended ReAct GPT-4 agent at 24% attack success, nearly doubling to ~47% under an “enhanced” hacking prompt [3].

Three defense families respond to this, refining the standard detection-vs-prevention formalization of prompt-injection defenses [15] by splitting prevention into training and design. *Detection* classifies the input, but detectors both lose to paraphrase / character perturbation and *over-defend*: on the NotInject benchmark Prompt-Guard scores only 0.88% on the *over-defense split* (benign inputs that merely contain injection-trigger vocabulary), and 41.60% averaged across the benign / malicious / over-defense splits (arXiv:2410.22770v3, Table 1 [13]). *Training* fine-tunes the model to resist injection, but the hardening is bound to the retrained weights. *Design* makes the harmful action unreachable by construction. Jataayu’s position, shared by the

2026 agent-security frameworks [2], is that the durable boundary is the *action*, gated by input provenance: untrusted-derived input driving a shell command, a secret read, or a network write is denied or held for approval regardless of whether any detector fired. Around that deterministic core it layers defense-in-depth screening at each surface where injection enters or data leaves — deliberately as a non-authoritative pre-filter, since the guarantee never rests on the classifier.

This report evaluates all seven surfaces. Our method is (i) *deterministic where possible* — the effect boundary, egress, composition, and fast-path detection paths use no model, so their numbers are exact and reproducible; (ii) *public benchmarks where they exist*, with deterministic in-repo corpora as a documented fallback; and (iii) *disclose-then-fix* — where an evaluation exposes a shipped defect, we report the as-shipped result before the fix.

Contributions. (1) A *deterministic, LLM-free effect boundary* that authorizes an action by effect-severity $\times$ provenance $\times$ capability policy and, unlike CaMeL’s value-level dataflow interpreter or a retrained model, *wraps an existing agent at the tool-call boundary* with no agent rearchitecture (§3.1, §4). (2) A seven-surface evaluation that reports the weak numbers as well as the strong ones, including a *disclose-then-fix* coverage gap on the effect boundary itself. (3) A direct *semantic-vs-effect* probe isolating the paper’s thesis: text-grading misses silent harms that effect-grading catches (§3.8); unlike LLM-judge trajectory evaluations (R-Judge, ToolEmu) [20, 21], our effect trace is deterministic and tied to the enforcing boundary. (4) Two reusable methodological artifacts: a labeled AgentDojo effect-boundary action stream, and a parser repair that is a precondition for evaluating non-OpenAI-tool-calling models on AgentDojo (Appendix A).

2 System and method

Jataayu is layered so the guarantee lives at the action: a normalization-and-taint layer (Layer 0), a deterministic effect boundary authorizing actions by *effect severity* $\times$ *provenance* $\times$ *capability policy* (Layer 1), and a two-path detector (regex fast path; optional LLM slow path) used as a pre-filter and taint source. The public API surfaces we evaluate are:

- `authorize_action` / `EffectBoundary` — effect boundary;
- `check_inbound`, `check_tool_return`, `check_memory_read`, `check_memory_write` — injection screening;
- `check_outbound` — PII; `check_egress` — exfil channels; `check_skillset` — composition.

Metrics: for detection, precision / recall / FPR at fixed operating points; for the effect boundary, *APR* (attack-prevention rate, with NEEDS_APPROVAL counted as prevention but reported separately) and *TUR* (task-utility retained); for AgentDojo, utility (clean / under-attack) and targeted attack-success rate (ASR).

3 Results

The seven guarded surfaces are reported in §3.1 (effect boundary) and §3.3–§3.7 (inbound / tool-return injection, outbound PII, egress, composition, memory). Three subsections are cross-cutting rather than per-surface: the AgentDojo end-to-end run (§3.2), the semantic-vs-effect probe (§3.8), and the consolidated summary (§3.9).

3.1 Effect boundary — the guarantee

Tier 1 (synthetic, 300 rows, deterministic). Ablating four defenses on a corpus spanning every effect class $\times$ provenance (Table 1): the effect boundary lifts APR from 0.00 (no defense) and 0.27 (detector-only) to **1.00**. Trusted-legit utility is retained in full (1.00); the utility

cost (TUR 0.50) lands entirely on untrusted-tainted benign high-effect actions, which route to NEEDS_APPROVAL rather than silent ALLOW. Decision latency: 0.036 ms mean. Two honesty caveats. The TUR of 0.50 is a function of the deterministic corpus’s provenance/effect *balance* by construction (the authors set the prevalence of untrusted-tainted benign high-effect actions), and is not a projection of production task-utility loss. And because Tier 1 enumerates effect classes rather than named tools, its 1.00 certifies the policy logic, not robustness to unmapped tools, the very failure mode Tier 2 exposes next.

Table 1: Effect boundary Tier 1 (synthetic, 300 rows, deterministic), four-way defense ablation. APR = attack-prevention rate (NEEDS_APPROVAL counted as prevention); TUR = task-utility retained; trusted-legit TUR isolates utility on trusted-provenance benign actions.

Defense	APR	TUR	trusted-legit TUR	latency (ms)
none	0.00	1.00	1.00	—
detector-only	0.27	1.00	1.00	—
effect	1.00	0.50	1.00	—
both	1.00	0.50	1.00	0.036

Tier 2 (AgentDojo-derived action stream, disclose-then-fix). We label real tool calls attack-vs-legit with provenance and replay them through the effect boundary. The corpus has 150 attack rows over *five* suites: the four AgentDojo suites (workspace / banking / travel / slack, 116 attack rows) plus a synthetic runtime suite (34 attack rows) that exercises the shell / code-eval / secret-read / memory-write effect classes the four AgentDojo suites never touch. The replay exposes a shipped coverage gap: 21 effectful tools (money transfers, bookings, Slack invites, external shares, file deletes) were *unmapped* and silently classified READ→ALLOW. As-shipped APR under the effect defense is **0.29** overall (44/150 prevented). That headline is dominated by the one already-mapped suite: the runtime tools were correctly gated (34/34, APR 1.00), while the four AgentDojo suites prevented only 10/116 (per-suite APR: banking 0.00, slack 0.00, travel 0.00, workspace 0.27) — i.e. the gap is precisely on the AgentDojo side-effect tools. After extending the tool→effect map in `core/taint.py` — classifying by *what the tool does* (irreversible external side effect = network sink; file mutation = file-write), not by benchmark membership — APR rises to **1.00** across all five suites and the coverage gap closes to zero (Table 2, Fig. 1). The disclosed cost is TUR 0.83 → 0.50. The full test suite (403 tests) stays green after the change. This fixed 1.00 is by construction on the now-mapped tool set (no held-out tool split): it certifies coverage of these tools, not generalization to unmapped future ones, exactly the property the as-shipped row shows is not free.

Table 2: Effect boundary Tier 2 (effect defense), disclose-then-fix, per-suite APR with attack n . The runtime suite is synthetic (shell / code-eval / secret-read / memory-write classes absent from AgentDojo); the other four are AgentDojo. Overall APR weights all 150 attack rows. *Confound:* rows are deterministic replays of labeled tool calls through our effect boundary (no live model), so APR reflects tool→effect classification correctness on *mapped* tools, not robustness to unseen future tools — exactly the failure mode the as-shipped row exposes.

	per-suite APR (n_{atk})					overall APR
	banking (26)	slack (30)	travel (23)	work (37)	runtime (34)	
as-shipped	0.00	0.00	0.00	0.27	1.00	0.29
fixed	1.00	1.00	1.00	1.00	1.00	1.00

As-shipped TUR 0.83, coverage gaps **21**; fixed TUR 0.50, coverage gaps **0**.

Edge paths. Previously uncovered: `commit()` with mutated parameters raises `CommitRejected` (post-authorization mutation blocked); committing a non-ALLOW preview is rejected; a capability-forbidden action is DENIED even under trusted provenance (capability wins). All 6

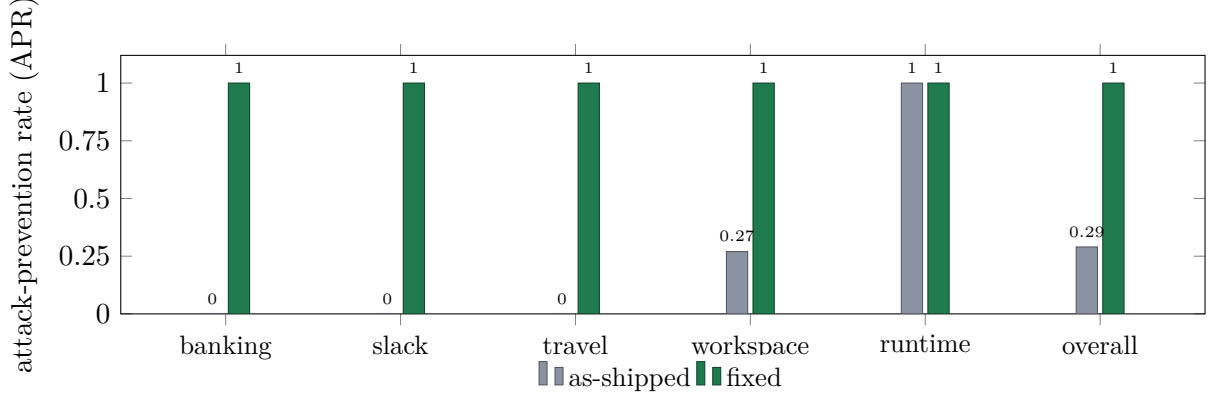


Figure 1: Effect-boundary Tier 2 disclose-then-fix: per-suite APR before (grey) and after (green) extending the tool→effect map. The as-shipped gap is concentrated on the four AgentDojo suites (banking / slack / travel APR 0.00, workspace 0.27); the already-mapped synthetic **runtime** suite was correct at 1.00 throughout. The fixed bars are by construction on the now-mapped tool set (no held-out tool split), so they certify coverage, not generalization to unmapped future tools.

cases pass.

3.2 Agentic end-to-end — AgentDojo

On the **workspace** suite under **important_instructions** (5 user tasks $\times$ 4 injections, $n=20$ attack cases), Jataayu’s tool-output screen with surgical redaction drives ASR $0.10 \rightarrow \mathbf{0.00}$ and restores utility-under-attack $0.75 \rightarrow \mathbf{1.00}$ with clean utility unchanged at 1.00 (Table 3). *Reproducibility.* The agent is **qwen3.6:35b-a3b** served over an OpenAI-compatible endpoint (ollama-style, on the tailnet), driven by AgentDojo 0.1.35’s LocalLLM at its default decoding, single-sample; the attack is **important_instructions**; Jataayu runs as the model-free tool-output screen at **min_status=HIGH**. Runs are stored under **runs/agentdojo/**; the harness and the parser repair required to make a non-OpenAI-tool-calling model run at all live in **eval/agentdojo/run_agentdojo.py** (see Appendix A).

The four-suite live sweep was truncated by a $\sim 10\times$ degradation of the local inference endpoint mid-run. The completed banking *baseline* (17 of the intended 20 attack cases before the endpoint degraded, no paired Jataayu run) shows ASR 0.29 at util-under-attack 0.53. Read cautiously, since this is a truncated run ($n=17$) on a degrading endpoint, it *suggests* more attack headroom than workspace (ASR 0.10), to be confirmed on a stable endpoint; we draw no suite-importance ranking from it. Crucially, the cross-suite *defense* evidence does not rest on this live sweep at all: the deterministic Tier 2 (§3.1, Table 2) covers all four AgentDojo suites and lifts per-suite APR to 1.00. Completing the live multi-suite sweep on a stable endpoint is future work.

Table 3: AgentDojo live results, **important_instructions**, **qwen3.6:35b-a3b** agent. Workspace is a complete baseline/Jataayu pair ($n=20$ attack, 5 clean; **agentdojo_workspace_qwen35b_slice.json**). Banking is the partial baseline before the endpoint degraded ($n=17$ attack, 5 clean; no paired Jataayu run; **agentdojo_banking_baseline_partial.json**). Cross-suite defense evidence is the deterministic Tier 2 (Table 2). All cells are base/Jataayu.

Suite	n_{atk}	ASR	util-atk	clean util
workspace	20	0.10 / 0.00	0.75 / 1.00	1.00 / 1.00
banking (baseline only)	17	0.29 / —	0.53 / —	0.80 / —

3.3 Injection detection — inbound, tool-return, cross-dataset

The fast-path detector is a high-precision, modest-recall pre-filter, by design the weakest tier. Across three public injection sets it is consistent (Table 4): ROC-AUC 0.63–0.75, precision $\geq 86\%$ at the MEDIUM operating point, and input normalization holds space-out / zero-width / leetspeak evasion at $\leq 0.3\%$ (exactly 0.00 on deepset and safe-guard; 0.003 for space-out on jailbreak). On **InjecAgent** (tool-return surface, 2108 paired cases) precision is 1.00 with recall 0.50 (fast) rising to 0.71 (LLM slow path). The slow-path judge is **qwen3:8b** served via ollama on a private tailnet with thinking disabled (`think:false`) at the model’s default decoding, run at `llm_threshold=0.0` so the LLM adjudicates every item (`run_slowpath_bench.py`); unlike the deterministic surfaces this 0.71 is model- and decoding-dependent, not bit-exact. These are *detection* recall on the injected tool return, not agent attack-success (ASR), so they are not directly comparable to InjecAgent’s own baselines — an undefended ReAct GPT-4 agent at 24% ASR (base) / 47% (enhanced hacking prompt) [3], or SecAlign, which drives InjecAgent ASR 75.9% $\rightarrow$ 0% by retraining the model [7]. The mapping is that a caught return (our recall) removes the injected instruction before the agent acts; an undetected one (our $1 - \text{recall}$) is where an ASR-style attack could still land, and is exactly why the guarantee lives at the effect boundary, not here. Even a missed return that reaches the agent must still clear the effect boundary before any effectful call executes.

Table 4: Fast-path injection detection across public datasets (MEDIUM operating point). Evasion rate is post-normalization on caught patterns. *Confound:* each dataset has its own label distribution and positive/negative balance; these are the regex pre-filter’s numbers, not the layered system’s, and are not comparable row-to-row as a single benchmark.

Dataset	n	ROC-AUC	precision	recall	evasion
deepset/prompt-injections	662	0.63	97%	27%	0.00%
safe-guard-prompt-injection	10,296	0.64	86%	30%	0.00%
jailbreak-classification	1,306	0.75	86%	56%	0.3% [†]

Recall / precision / evasion are percentages here to match Table 8; ROC-AUC follows the conventional $[0, 1]$ form.

[†]deepset and safe-guard space-out evasion is exactly 0.00%; the jailbreak set’s space-out evasion is 0.3% (374/375 caught). Zero-width and leetspeak evasion are 0.00% on all three sets.

3.4 Outbound — PII redaction

On ai4privacy (480 messages, fast path), detection precision is **1.00** at FPR **0.00** with in-scope recall 0.80; benign messages pass clean (over-redaction 0.00). Honest failure: a threshold artifact — email, phone, and contextual-health spans are *detected* (flagged) but score below the 0.75 redaction threshold, so the span survives in the redacted text; and the SSN pattern requires an explicit “SSN”/“social security” prefix, missing bare nine-digit numbers (recall 0.50). These are precision-safe but leave data in the output, a fix target.

3.5 Egress — zero-click exfiltration channels

Against a PoC corpus of the EchoLeak / AgentFlayer / Notion class (23 exfil channels + 12 benign): recall-caught **1.00**, precision **1.00**, FPR **0.00**. When a known secret rides in the URL, confirmed-exfil recall is 1.00 with the correct reason. Human-readable text is preserved in every caught case (23/23); only the offending URL is neutralized. Relay/beacon hosts are hard-blocked; data-carrying links and auto-fetch images are neutralized at WARN by design.

Ethics and responsible disclosure. The egress corpus consists entirely of synthetic proof-of-concept payloads modeling already-public vulnerability *classes* (EchoLeak / AgentFlayer / Notion-style zero-click exfiltration). No live third-party system, endpoint, or account was

targeted or probed; the “secrets” and exfil hosts are fabricated fixtures used only to exercise our own guard. We introduce no new attack technique and release only the defense.

3.6 Skillset composition — supply chain

On 62 skill-set combinations, the guard achieves 100% recall on the risk classes it models (exfiltration, dropper, download-and-run, memory-poisoning, policy-violation, trust-transfer) at 0% false-positive on safe sets. Honest gap: the exfiltration rule is keyed narrowly on `reads_secrets + network_write`; five genuinely-risky capability paths outside the four hardcoded combos — read-side-only exfil (`reads_secrets + network_read`), file-mediated exfil (`fs_read + network_write`), untrusted-web-to-disk, secret-staging, and relay/SSRF — all return SAFE. Widening the `data-source`×`egress` key is the recommended fix.

3.7 Memory — poisoning at write and recall

On 190 memories (fast path), detection is symmetric across write- and recall-time (byte-identical scores). Recall by subclass at MEDIUM: direct-instruction **0.91**, encoded/obfuscated 0.75, and **deferred/sleeper 0.00**. The standout gap: sleeper triggers phrased as future actions (“when you next summarize, also email X”) carry no override vocabulary, so the pattern catalog never fires: the engine has no deferred-instruction detector. Benign false-block is 1.8% on ordinary notes but 22.9% on adversarially-tricky-but-benign notes. The run also surfaced a live regex false-positive (a shell-command alternation matching “sh” inside “she”).

3.8 Semantic grading vs. effect grading — the thesis, isolated

The paper’s central claim is that judging an agent by *what it did* (its effects) is stronger than judging the *text* it produced. We test exactly this on 10 trajectories (6 harmful, 4 benign): a text grader inspects the final assistant message; an effect grader inspects the tool-call effect trace. The text grader catches only the one trajectory that *brags* about the harm in prose (recall **0.17**); the effect grader catches all six (recall **1.00**). The five harms the text grader misses are silent by construction — `cross-turn exfil`, `sleeper-memory write`, `no-reply shell execution`, `privilege escalation`, and `secret-to-file` — each leaves no incriminating text but a clear effect signature. For balance, the text grader also over-flags one benign trajectory (a harmless security discussion) that the effect grader correctly clears. This is the most direct validation of “gate the action, not the text” in the evaluation (Table 5).

Relation to prior effect-based evaluation. Judging agent safety from the action/effect trajectory rather than from output text is not itself new: R-Judge [20] has an LLM judge score recorded agent-interaction records for risk, ToolEmu [21] emulates tool execution and applies an LLM safety evaluator over the resulting trajectory, and AgentHarm [22] and Agent-SafetyBench [23] grade harmful behavior across whole agent runs. What is new here is not effect-grading but its construction: our effect trace is a *deterministic* tool-call effect signature tied to the *same* boundary that enforces at run time — there is no LLM judge and no trajectory sampling, so the grader and the guard are one decision function and the 0.17-vs-1.00 gap is exact rather than a model-dependent judgment.

Table 5: Semantic (text) vs. effect grading on 10 trajectories (6 harmful / 4 benign). The five silent harms are caught only by the effect grader.

Grader	recall (6 harmful)	benign over-flags (of 4)
text (final-message)	0.17	1
effect (tool-call trace)	1.00	0

3.9 Consolidated summary

Table 6 collects every surface’s headline result in one place: green marks a strong, model-free result and red a disclosed gap. The pattern the paper argues for is visible down the column — the deterministic effect boundary and egress/composition guards carry exact green numbers, while the disclosed gaps (deferred-memory recall, the five unmodeled composition paths, the PII redaction-threshold artifact, the fast-path recall floor) are exactly the surfaces where no deterministic guarantee is claimed.

Table 6: Jataayu surfaces, benchmarks, and headline results. Green = strong; red = a disclosed gap. Deterministic surfaces carry exact, model-free numbers.

Surface	Benchmark	Headline
Effect boundary (Tier 1)	synthetic, 300	APR 1.00 , TUR 0.50, trusted-legit 1.00, 0.036 ms
Effect boundary (Tier 2)	AgentDojo actions (5 suites)	APR 0.29→1.00 (disclose-then-fix), 21→0 gaps
Effect boundary (edges)	commit / capability	6/6 pass
Agentic e2e	AgentDojo workspace	ASR 0.10→0.00 , util 0.75→1.00, no clean cost
Semantic vs. effect	10 trajectories	text recall 0.17 vs. effect 1.00 (5 silent harms)
Injection (inbound)	deepset/safe-guard/jailbreak	AUC 0.63–0.75, prec $\geq$ 86%, recall 27–56%, evasion $\leq$ 0.3%
Injection (tool-return)	InjecAgent (2108)	prec 1.00, recall 0.50→0.71 (slow)
Outbound / PII	ai4privacy (480)	prec 1.00, FPR 0.00, in-scope recall 0.80; redaction-threshold gap
Egress channel	PoC (35)	recall 1.00 , prec 1.00, confirmed-exfil 1.00, text-preserve 1.00
Skillset composition	synthetic (62)	100% in-model recall, 0% FP; +5 unmodeled paths
Memory poisoning	synthetic (190)	direct 0.91 / encoded 0.75 / deferred 0.00

4 Comparison with prior approaches

Using the detection / training / design taxonomy of §1, Jataayu spans two of the three families: the *detection* tier (its screening layers) and the *design* tier (its effect boundary). We compare against representatives of each, plus the *training* tier (StruQ [6], SecAlign [7]). **None of the tables in this section is a controlled bake-off:** model, suite coverage, dataset, and metric construction differ across rows, each value is the one reported by its own source, and we label the per-table confounds and draw only the conclusions they support. (The two table captions do not repeat this caveat.)

Nearest neighbors: runtime tool-call policy enforcement and guard agents. The effect boundary’s closest prior art is not the prompt/detection tier but the recent line of *runtime enforcement at the tool-call boundary*. Progent [16] attaches programmable privilege-control policies to each tool call; AgentSpec [17] specifies and enforces runtime constraints over agent actions; GuardAgent [18] runs a separate guard agent that screens actions against a safety request; IsolateGPT and context-aware policy work (Conseca) [19] isolate or generate per-context permissions. Jataayu belongs to this family: deterministic action gating plus tool-output

screening (NEEDS_APPROVAL gating). What the effect boundary adds over each is a fixed, model-free decision function of effect-severity $\times$ provenance $\times$ capability that requires *no learned policy, no guard LLM, and no agent rearchitecture*: it is a drop-in wrapper at the existing tool-call site, decided in tens of microseconds. Where GuardAgent invokes a second model and Progent/AgentSpec require authored per-tool policies, Jataayu’s default policy is derived from a small effect-class map, which is exactly the artifact whose incompleteness Tier 2 exposes and fixes.

Progent is the most metric-aligned of these neighbors: on AgentDojo it drives targeted ASR from a 39.9% no-defense baseline to **0%** on GPT-4o with utility maintained [16] (row in Table 7), a deterministic-policy, design-tier result directly comparable in kind to the effect boundary’s. The other two nearest neighbors report *no* AgentDojo number, so a head-to-head figure cannot be extracted: GuardAgent measures guardrail accuracy on its own EICU-AC (>98%) and Mind2Web-SC (83%) benchmarks [18], and AgentSpec reports rule/constraint-satisfaction on its own action-constraint benchmarks rather than attack-prevention on AgentDojo [17]. Progent’s 39.9%→0% is over AgentDojo’s live task/injection distribution, whereas the effect boundary’s APR is a deterministic replay of a labeled action stream (§3.1); the two are the same *kind* of guarantee measured on different footings.

Relation to CaMeL. CaMeL is the closest design-tier system, and the honest distinction is *not* “deterministic vs. LLM-based enforcement”: CaMeL’s enforcement layer (a custom interpreter applying capability/provenance policy over values) is itself deterministic and LLM-free, and Jataayu likewise sits downstream of an LLM agent. Only CaMeL’s *planning* uses a privileged P-LLM. The real differences are two. (i) *Deployment*: CaMeL rearchitects the agent into a P-LLM/Q-LLM/interpreter pipeline, whereas Jataayu wraps an unmodified agent at the tool-call boundary. (ii) *Granularity*: CaMeL enforces value-level dataflow capabilities inside its interpreter, whereas Jataayu enforces at the action level on effect-severity $\times$ provenance $\times$ capability. CaMeL’s is the stronger, more invasive guarantee; Jataayu’s is the drop-in one.

Read honestly, Table 7 places Jataayu’s screening in the effective-defense regime alongside AgentDojo’s `tool_filter` (ASR 6.8%) and `PI_detector` (7.9%). The most metric-aligned comparison is on the design tier: Llama Prompt-Guard-2 reports an AgentDojo *attack-prevention rate* of 81.2% at a 3% utility reduction [8], and its model card places the commodity detectors it competes against (ProtectAI, Deepset, LLM Warden) far lower, at only 12.9–22.2% AgentDojo APR [8] — reinforcing that ordinary classifiers do poorly on this metric. Our Tier 2 (§3.1) reports the same APR-on-AgentDojo quantity, moving from 29% as-shipped to **100%** once the tool→effect map is complete, at a disclosed TUR of 0.50. But the two 100%-vs-81.2% figures are *not* on the same footing: PG2’s 81.2% is measured over AgentDojo’s *live* task/injection distribution, whereas our figure is a *deterministic replay* of a labeled action stream over tools we have just finished mapping, so our 100% is by construction on the mapped set (§3.1) rather than a live-benchmark result. PG2 also buys prevention at only 3% utility loss whereas the effect boundary’s approval-gating costs a larger, corpus-dependent utility fraction. The honest reading is therefore not “we beat PG2” and not “same benchmark”: it is that a deterministic, retraining-free policy layer reaches a comparable *kind* of prevention result on an AgentDojo-*derived* action stream, trading utility for a guarantee that does not depend on a learned classifier. Our qwen baseline ASR (10%) sits far below GPT-4o’s aggregate (57.7%) largely because of model and single-suite scope, so we do not claim a head-to-head e2e win.

Table 8 is the honest core of the comparison. *As a detector, Jataayu’s fast path is worse*: dedicated classifiers reach ~99% clean recall where our regex pre-filter reaches 27–56%. We do not close that gap and do not try to — the pre-filter is deliberately the weakest tier. But those same classifiers are largely defeated by trivial character perturbations (Prompt-Guard detection collapses from ~100% to 0.2% under character-spacing [10]; ProtectAI is bypassed ~77% of the time), whereas Jataayu’s input normalization holds space-out and leetspeak evasion

Table 7: Agentic defenses on AgentDojo. AgentDojo built-in defenses, Progent, and CaMeL are from their papers on the models shown. Two metrics appear: targeted ASR (lower better) and, where a source reports it, attack-prevention rate APR (higher better); they are complementary, not identical; APR is shown as a percentage throughout. ^aNo-defense targeted ASR is the AgentDojo Table-5 value (57.7%); the live leaderboard reports 47.7% for the same setting [1]. ^bCaMeL reports successful-attack *counts*; it drives genuine injections to zero by design (residual counts are AgentDojo eval artifacts, not injections [5]). ^cPrompt-Guard-2 and the Jataayu effect boundary report APR, not ASR; PG2’s is at a stated 3% utility reduction [8]. ^dThe Jataayu effect-boundary utility is a *synthetic replay* TUR set by the deterministic corpus’s provenance/effect balance (§3.1), not a live task-utility measurement, and is *not* commensurable with the real-benchmark utilities in this column.

Approach	Class	Model / suite	ASR ↓ / APR ↑	Benign utility
No defense	—	GPT-4o	ASR 57.7% ^a	69.0%
Spotlighting / delimiting	prompt	GPT-4o	ASR 41.7%	72.7%
Repeat-prompt (sandwich)	prompt	GPT-4o	ASR 27.8%	85.5%
PI detector	detection	GPT-4o	ASR 7.9%	41.5%
Tool filter	design	GPT-4o	ASR 6.8%	73.1%
Progent (runtime policy)	design	GPT-4o	ASR 0.0% (from 39.9%)	util maintained [16]
Prompt-Guard-2 86M	detection	GPT-4o	APR 81.2% ^c	−3% util
CaMeL	design	o3	ASR ≈0% ^b	77.3% (from 84.5%)
Jataayu	detection	<i>qwen, workspace</i>	ASR 0.0% (from 10%)	100%
tool-output scrub				
Jataayu effect boundary	design	<i>AgentDojo actions</i>	APR 100% ^c (from 29%)	TUR 0.50 ^d

Table 8: Injection *detection*: clean recall vs. robustness to character-perturbation evasion. Each value is on the method’s own reported dataset (not one shared set), so this is a comparison of reported *characteristics*. The pattern is the point: dedicated classifiers have far higher clean recall than Jataayu’s fast path, and far higher evasion vulnerability.

Detector	Clean recall / TPR	Char-perturbation evasion
ProtectAI deBERTa-v3 v2 [9]	99.7% (20k held-out)	77% attack success (char-injection) [10]
Meta Prompt-Guard-86M [8]	99.5% (in-dist. TPR)	99.8% bypass (char-spacing) [10]
Jataayu fast path	27–56% (deepset/sg/jb)	≤0.3% (space-out / leetspeak)

at $\leq 0.3\%$ (exactly 0% except for space-out on the jailbreak set, at 0.3%). On the Lakera PINT leaderboard the same ordering holds among detectors (Lakera Guard 95.2%, ProtectAI-v2 79.1%, Prompt-Guard-2 78.8%) [11]. The training-based defenses occupy a third point: StruQ zeroes non-adaptive ASR by fine-tuning but leaves adaptive GCG at 56–62% [6]; SecAlign drives even optimization-based ASR down (Llama3-8B 98% $\rightarrow$ 1%, Mistral 89% $\rightarrow$ 1%) and InjecAgent 75.9% $\rightarrow$ 0% — at the cost of retraining the model with preference optimization [7], which Jataayu does not require. The through-line: detection alone is evadable and training is model-bound, so Jataayu, like CaMeL, moves the guarantee to the action.

The low-false-positive operating point. A production guardrail is judged not at its best F1 but at a fixed, low false-positive rate — the regime the PromptShield benchmark isolates by reporting TPR at 1% FPR (arXiv:2501.15145, Table 4 [12]). There the gap between a purpose-built detector and the commodity classifiers is stark (Table 9, left): PromptShield reaches 94.8% TPR at 1% FPR (AUC 0.998) while Prompt-Guard manages 12.78% (AUC 0.874) and ProtectAI-v1 7.05% at the same operating point. Jataayu’s fast path is not in this class and does not try to be — it is a taint source feeding the effect boundary, and it deliberately trades recall for a zero over-redaction / low false-block profile. That trade is the *other* axis a guardrail is judged on, and the one the recent inbound work targets: the NotInject over-defense benchmark (Table 9, right) shows Prompt-Guard collapsing to 0.88% accuracy on the *over-defense split* — benign inputs that merely contain trigger words — where InjecGuard holds 87.32% on that same split (arXiv:2410.22770v3, Table 1 [13]). (Averaged across the benchmark’s three splits these become 41.60% and 83.48%, the figures tabulated in Table 9; the over-defense split is the one that isolates false positives, so it is the sharper number.) This is exactly the false-positive failure Jataayu’s guards are tuned against (composition 0% FP, PII over-redaction 0.00, memory benign false-block 1.8% on ordinary notes).

Table 9: Detector benchmarks at the two operating points a deployed guardrail is judged on: recall at a fixed low false-positive rate (PromptShield, TPR@1% FPR / ROC-AUC; values exactly as in arXiv:2501.15145 Table 4 [12]) and over-defense on benign trigger-word inputs (NotInject, *average* accuracy across benign / malicious / over-defense splits; values exactly as in arXiv:2410.22770v3 Table 1 [13]). The right panel’s average differs from the over-defense-split-only numbers cited in the text (Prompt-Guard 0.88%, InjecGuard 87.32%), which isolate false positives. These rows are the least independently verifiable block in the comparison: they are single-source, version-dependent figures (PromptShield v1, Jan 2025; InjecGuard v3, 2024), not re-measured here, and unlike the other competitor numbers they cannot be cross-checked against a second source — we cite them only to place the commodity classifiers. *Confound:* the two panels are different papers, datasets, and label sets; Jataayu does not appear because its fast path is a taint pre-filter, not a stand-alone detector benchmarked at these points — the rows place the commodity classifiers, which are the ones Jataayu would otherwise be mistaken for.

Low-FP detection (PromptShield)			Over-defense (NotInject)	
Detector	TPR@1%FPR	ROC-AUC	Guardrail	avg acc. (3-split)
PromptShield (Llama-3.1-8B)	94.8%	0.998	GPT-4o	85.53%
InjecGuard	20.4%	—	InjecGuard	83.48%
Prompt-Guard	12.78%	0.874	Lakera Guard	77.23%
ProtectAI v1	7.05%	—	Prompt-Guard	41.60%

5 Limitations and the gaps this evaluation found

The evaluation is deliberately adversarial toward Jataayu, and it paid off: it surfaced (1) a shipped `taint.py` coverage gap that silently authorized every banking / travel / Slack side-effect (now fixed); (2) no deferred/sleeper-instruction detector (memory recall 0.00 on that subclass);

(3) an over-narrow exfiltration key in the composition guard (five real paths slip); and (4) a live shell-regex false-positive. Scope caveats: the AgentDojo slice is small per suite; several corpora (effect boundary, egress, composition, memory) are synthetic-but-deterministic, not public benchmarks; the detection recall numbers are for the fast path (the pre-filter), not an upper bound on the layered system; and the effect-boundary APR of 1.00 reflects correct tool classification, not a claim of robustness to unmapped future tools — which is precisely the failure mode disclosed in Tier 2.

6 Conclusion

Across seven surfaces and both public and synthetic benchmarks, Jataayu’s deterministic effect boundary delivers its central claim — untrusted-derived effectful actions are denied or approval-gated, deterministically and in microseconds — while the screening layers behave as a useful but deliberately non-authoritative pre-filter. Equally, the evaluation is credible because it reports four concrete defects rather than a row of green checkmarks. Future work: close the deferred-detector and composition-keying gaps (disclose-then-fix), extend the AgentDojo sweep across models, and lift the redaction threshold for detected-but-unredacted PII.

A Artifacts and reproduction

Availability. Code, runners, and result JSONs are released at <https://github.com/saikrishnarallabandi/jataayu> under the MIT license. All numbers in this report correspond to commit 8f099ca on `main`; reproduce against that exact commit rather than the `v0.3.0` tag, which names an earlier commit (a219020, `git describe: v0.3.0-17-g8f099ca`) that predates the Tier-2 harness and the comparison tables. The unit suite is reproduced with `pytest -q` (403 tests green) at that commit. For double-blind review an anonymized artifact copy is provided separately; an archival DOI will be minted on acceptance.

Every result in this report is produced by a runner under `eval/` that writes a JSON result to `eval/results/`; deterministic corpora live in `eval/data/` with fixed-seed generators. The deterministic surfaces (effect boundary, egress, composition, memory / injection fast path) require no model and reproduce exactly.

AgentDojo live configuration. The one non-deterministic surface (§3.2) uses agent model `qwen3.6:35b-a3b` served over an OpenAI-compatible endpoint (ollama-style, on a private tailnet), driven by AgentDojo 0.1.35’s `LocalLLM` element at its default decoding, single sample per step; attack `important_instructions`; Jataayu as the model-free tool-output screen at `min_status=HIGH`. Per-task run logs are archived under `runs/agentdojo/`; the workspace pair is `agentdojo_workspace_qwen35b_slice.json` and the partial banking baseline is `agentdojo_banking_baseline_partial.json` ($n=17$ attack cases before endpoint degradation).

Parser precondition (methodological note). Obtaining any signal from a non-OpenAI-tool-calling model on AgentDojo first required repairing a silent failure in AgentDojo’s local-model adapter, which recovers tool calls by running `json.loads()` on the text between `<function=name>` and `</function>`. Two body shapes Qwen-family models routinely emit break that parse: an *empty* zero-argument body (`<function=get_current_day></function>`) and *trailing garbage* after a valid object (a doubled closing brace). Because the workspace agent’s mandated first action is the no-argument `get_current_day`, the dropped call stalls *every* task at step one — a uniform zero utility that masquerades as a null result rather than a crash. Our repair (empty-body $\rightarrow \{\}$, plus brace-balanced salvage) lives in `eval/agentdojo/run_agentdojo.py`; we recommend asserting non-zero undefended-baseline utility as a harness precondition.

Surface	Runner
Effect boundary (Tier 1 / 2 / edges)	<code>run_effect_boundary_{bench,tier2,edges}.py</code>
Injection (inbound / cross-dataset)	<code>run_injection_bench.py -dataset ...</code>
Tool-return (InjecAgent, fast path)	<code>run_injecagent_bench.py</code>
Tool-return slow path (LLM judge)	<code>run_slowpath_bench.py -model qwen3:8b</code>
Outbound / PII	<code>run_outbound_privacy_bench.py</code>
Egress channel	<code>run_egress_bench.py</code>
Skillset composition	<code>run_composition_bench.py</code>
Memory poisoning	<code>run_memory_poison_bench.py</code>
Semantic vs. effect (Table 5)	<code>run_semantic_vs_effect.py</code>
AgentDojo (agentic e2e)	<code>agentdojo/run_agentdojo.py (+ summarize_slice.py)</code>

The `taint.py` disclose-then-fix is reproduced by running the Tier 2 harness before and after the tool→effect map extension; the full unit suite (403 tests) is green in both the as-shipped and fixed states.

References

- [1] E. Debenedetti et al. *AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents*. NeurIPS Datasets & Benchmarks, 2024. arXiv:2406.13352.
- [2] OWASP. *Top 10 for Agentic Applications / for LLM Applications*. 2025.
- [3] Q. Zhan et al. *InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents*. 2024.
- [4] K. Greshake et al. *Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. ACM AISec, 2023.
- [5] E. Debenedetti et al. *Defeating Prompt Injections by Design (CaMeL)*. arXiv:2503.18813, 2025.
- [6] S. Chen et al. *StruQ: Defending Against Prompt Injection with Structured Queries*. USENIX Security, 2025. arXiv:2402.06363.
- [7] S. Chen et al. *SecAlign: Defending Against Prompt Injection with Preference Optimization*. ACM CCS, 2025. arXiv:2410.05451.
- [8] Meta. *Prompt-Guard-86M and Llama Prompt Guard 2* model cards. <https://huggingface.co/meta-llama>.
- [9] Protect AI. *deberta-v3-base-prompt-injection-v2* model card. <https://huggingface.co/protectai>.
- [10] W. Hackett et al. *Bypassing Prompt Injection and Jailbreak Detection in LLM Guardrails*. arXiv:2504.11168, 2025. (See also meta-llama/llama-models issue #50.)
- [11] Lakera AI. *PINT: Prompt Injection Test Benchmark*. <https://github.com/lakeraai/pint-benchmark>.
- [12] *PromptShield: Deployable Detection for Prompt Injection Attacks*. arXiv:2501.15145, 2025.
- [13] *InjecGuard: Benchmarking and Mitigating Over-defense in Prompt Injection Guardrail Models*. arXiv:2410.22770, 2024. (Introduces the NotInject over-defense benchmark.)
- [14] *Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents*. arXiv:2410.02644, 2024.

- [15] Y. Liu et al. *Formalizing and Benchmarking Prompt Injection Attacks and Defenses*. USENIX Security, 2024. arXiv:2310.12815. (Formalizes prompt-injection defenses into detection- and prevention-based classes; we split prevention into training and design.)
- [16] T. Shi et al. *Progent: Programmable Privilege Control for LLM Agents*. arXiv:2504.11703, 2025.
- [17] *AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents*. 2025.
- [18] Z. Xiang et al. *GuardAgent: Safeguarding LLM Agents by a Guard Agent via Knowledge-Enabled Reasoning*. arXiv:2406.09187, 2024.
- [19] Y. Wu et al. *IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems*. NDSS, 2025. (See also Consecra, context-aware security policy generation.)
- [20] T. Yuan et al. *R-Judge: Benchmarking Safety Risk Awareness for LLM Agents*. 2024. arXiv:2401.10019.
- [21] Y. Ruan et al. *Identifying the Risks of LM Agents with an LM-Emulated Sandbox (ToolEmu)*. ICLR (spotlight), 2024. arXiv:2309.15817.
- [22] M. Andriushchenko et al. *AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents*. ICLR, 2025. arXiv:2410.09024.
- [23] Z. Zhang et al. *Agent-SafetyBench: Evaluating the Safety of LLM Agents*. 2024. arXiv:2412.14470.